Shardwise: Measurement Pitfalls in Repartitioning Small CPU Language Models

B.Dinesh
Amrita School of Artificial Intelligence, Coimbatore
Amrita Vishwa Vidyapeetham, India
cb.ai.u4aid24107@cb.students.amrita.edu

Jagath Varun.V
Amrita School of Artificial Intelligence, Coimbatore
Amrita Vishwa Vidyapeetham, India
cb.ai.u4aid24121@cb.students.amrita.edu

K.Charan Teja
Amrita School of Artificial Intelligence, Coimbatore
Amrita Vishwa Vidyapeetham, India
cb.ai.u4aid24124@cb.students.amrita.edu

K.Praveen
Amrita School of Artificial Intelligence Coimbatore
Amrita Vishwa Vidyapeetham, India
cb.ai.u4aid24166@cb.students.amrita.edu

S. Manimaran*
Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
s_manimaran@cb.amrita.edu

***Abstract*—Splitting a language model that fits on each available machine creates a serving choice rather than a capacity requirement. We study that choice with Shardwise, a CPU runtime that partitions a decoder-only model into contiguous layer ranges, selects placements with latency (sum) or throughput (bottleneck) objectives, re-partitions on slowdown or worker loss, and rebuilds state by replaying token history. In 65 real-model runs (939 requests) of Qwen3-0.6B on two virtual machines, throughput-oriented placement raised aggregate output throughput by 80.5% at two concurrent requests, while latency-oriented placement was 65.4% faster for one request under injected 120 ms delay. Adaptation raised fault-phase throughput by 25.0% under injected compute degradation but only 6.8% over the whole run. A migration-payback gate showed no advantage over hysteresis, and measured transitions exceeded their forecasts by 29–76%. Queueing accounted for 99.2% of a synthetic call-minus-compute residual. Single-repetition local checks found replica baselines competitive. Eighteen replay cases reproduced reference outputs while showing token gaps of up to about ten seconds. Baseline choice, transition accounting and telemetry contamination limit what small deployments can support.**

***Index Terms*—LLM inference, pipeline parallelism, CPU serving, performance measurement, recovery**

I. Introduction

A small language model can fit on each of several available machines. Splitting its layers is then one serving option, and complete-model replication is another. Pipelining lets independent requests overlap across stages, but a single autoregressive request traverses every active stage for each token. Moving layer boundaries adds a second tradeoff: a better steady-state placement may arrive too late to repay its reconfiguration cost.

Shardwise makes these costs inspectable. A Go controller places contiguous decoder-layer ranges, Python workers execute them over gRPC, and a router tracks independent request histories and rebuilds KV caches after a generation change. The question is how measurement and baseline choices change the conclusions drawn from a small running service.

We make four contributions:

- An audited empirical study of 65 real-model runs and 939 requests. Failures stay in all denominators, and whole-run and fault-phase metrics are reported side by side.
- An exact ordered-partition recurrence and a finite-concurrency capacity envelope. The envelope explains when each objective applies and corrects a two-worker heuristic in the earlier runtime.
- Payback conditions for fixed and remaining-work horizons, and an empirical check showing that the payback gate is only as accurate as its transition forecast.
- Position-controlled replay checks and queue instrumentation that separate replay correctness from client-visible stalls, and queue waiting from link cost in telemetry.

Exact optimality under the model does not imply optimal placement on the real runtime. We do not claim a new migration, recovery, or partitioning principle.

The remainder of this paper is organized as follows. Section II reviews distributed and adaptive LLM inference systems. Section III presents the Shardwise architecture, adaptive placement objectives, and migration-payback model. Section IV describes the testbed, workloads, baselines, metrics, and recovery checks. Section V reports and discusses the experimental results, and Section VI concludes the paper.

II. Related Work

Pipeline parallelism divides an ordered model into stages so that independent requests can overlap, while contiguous graph partitioning assigns adjacent operations to workers [1], [15]. These methods are often motivated by models that exceed the memory of one device. Shardwise instead studies a model that

fits on every participating machine. Distribution is therefore optional, and its value must be established against complete-model replication and unsplit execution.

Collaborative inference systems explore several forms of device aggregation. Petals distributes transformer blocks across volunteer devices and provides fault-tolerant collaborative inference [2]. EdgeShard formulates contiguous layer placement under device and memory constraints and distinguishes sequential latency from pipeline throughput [3]. Galaxy combines tensor parallelism with heterogeneous edge execution [4], while TPI-LLM targets memory-constrained mobile devices [5]. Prima.cpp coordinates heterogeneous home computers for large-model inference [6]. These systems primarily address capacity or accelerator utilization; Shardwise focuses on measurement validity when a small CPU model can run without distribution.

Adaptive serving systems change placements as resources or workloads vary. Llumnix migrates model-serving workloads to reduce fragmentation and interference [7]; ServerlessLLM reconstructs inference state from token history [8]; and SpotServe re-parallelizes services under preemption [9]. DynoPipe moves pipeline boundaries across heterogeneous edge and cloud GPUs [10], whereas Kafetzis et al. include migration overhead when repartitioning attention heads [16]. DiSCo and STAR admit migration only when expected remaining decode savings can repay the handover cost [11], [12]. These approaches motivate the payback analysis used here, but their GPU-oriented assumptions do not directly determine CPU behavior.

Failure handling introduces a related tradeoff between readiness and resource cost. ShadowLLM maintains hot standbys for heterogeneous edge serving [13], while EASTER learns partitions designed to tolerate device loss [14]. Shardwise instead reconstructs request state by replaying accepted token history after a generation change. This avoids a continuously active standby but can create a long client-visible stall, so output correctness and recovery delay must be evaluated separately.

Table I summarizes the closest comparisons. Prior work generally evaluates placement, adaptation, or recovery as the primary mechanism. Shardwise combines all three in an inspectable CPU runtime and emphasizes measurement choices: failures remain in denominators, fault-phase and whole-run throughput are reported together, transition time is charged to adaptation, and queue waiting is separated from link estimates. These choices follow established benchmarking guidance [17] and define the main empirical distinction of this work.

III. System Architecture and Adaptive Placement

A. Execution, recovery and deployment

Workers hold half-open ranges of decoder layers. The first active stage embeds tokens and the last applies normalization and the output head. Each worker serializes compute with one lock. Request threads can overlap between stages, but there is no continuous batching or speculative decoding. In cached mode, each step sends one new position per request.

Every assignment carries a generation number, and workers reject forwards from other generations. On relayout, missing cache, or an unreachable worker, the router obtains the surviving layout and replays the full accumulated token history under a fresh request identifier. Accepted output tokens remain in the history. Recovery requires enough surviving memory to hold the assigned model and does not protect the router or controller machine.

Node agents report compute speed and link observations. The controller rejects snapshots older than 5 s or more than 2 s in the future, and holds voluntary movement when no usable snapshot exists. Application timing historically used RPC time minus worker compute time, which also includes time waiting for the worker's compute lock. The revision reports lock wait separately and offers an opt-in queue-corrected residual. That residual still contains serialization, dispatch and transport, so it is not pure network round-trip time.

B. Placement and the concurrency envelope

For a nonempty worker range $[a, b)$ with L layers, the cost model is

$$d_i(a, b; C) = \frac{(b-a)\, c(C) + e\, \mathbf{1}_{a=0} + h\, \mathbf{1}_{b=L}}{\max(s_i, 0.01)} + \ell_i, \qquad (1)$$

where $c(C)$ is per-layer compute time at context C, e and h are embedding and head costs, s_i is relative speed, and ℓ_i is the link cost into worker i ($\ell_0 = 0$). Empty workers cost zero. The model omits memory feasibility, transfer–compute overlap and prefill/decode interference. For a placement π, the sequential cost is $P = \sum_i d_i$ and the bottleneck cost is $B = \max_i d_i$. The latency objective is $J_{\mathrm{lat}} = P$ and the throughput objective is $J_{\mathrm{thr}} = B$. The ordered dynamic program

$$F_o(w, j) = \min_{0 \le a \le j} \phi_o\big(F_o(w-1, a),\, d_{w-1}(a, j)\big) \qquad (2)$$

uses $\phi_{\mathrm{lat}}(x, y) = x + y$ and $\phi_{\mathrm{thr}}(x, y) = \max(x, y)$. Its state space has $O(WL)$ entries with $O(L)$ choices each, so it runs in $O(WL^2)$ time and $O(WL)$ space. Every feasible assignment ends with a unique boundary, and replacing its prefix with an optimal one cannot worsen the total, so the recurrence is exact over all ordered contiguous assignments, including empty workers. An exhaustive checker confirms both objectives on 1,000 random small instances.

For Q non-speculative closed-loop requests, ideal aggregate throughput obeys

$$X_Q \le \min\Big(\frac{1}{B}, \frac{Q}{P}\Big), \qquad J_Q = \max\Big(B, \frac{P}{Q}\Big). \qquad (3)$$

The first bound follows from per-stage utilization. The second follows from Little's law [18] applied to a closed network whose cycle time is at least P [19]. At $Q = 1$ the relevant cost is P, so the sum objective is correct. The bottleneck objective matters once $Q \ge P/B$; two balanced stages therefore need $Q = 2$. These are capacity bounds, not throughput predictions.

The historical automatic policy chose the sum objective for $Q \le 1$ and the bottleneck objective otherwise. This rule fails with more workers. Nine unit-cost layers on three workers,

TABLE I
COMPARISON WITH REPRESENTATIVE DISTRIBUTED LLM SYSTEMS.

System	Platform	Placement	Adaptation	Recovery	Main emphasis
Petals [2]	Consumer devices	Blocks	Limited	Yes	Collaborative inference
EdgeShard [3]	Edge devices	Layers	No	No	Latency and throughput
DynoPipe [10]	Edge–cloud GPUs	Stages	Yes	No	Dynamic boundaries
DiSCo/STAR [11], [12]	Accelerators	Requests	Yes	Partial	Migration payback
Shardwise	CPU VMs	Layers	Yes	Replay	Measurement and adaptation

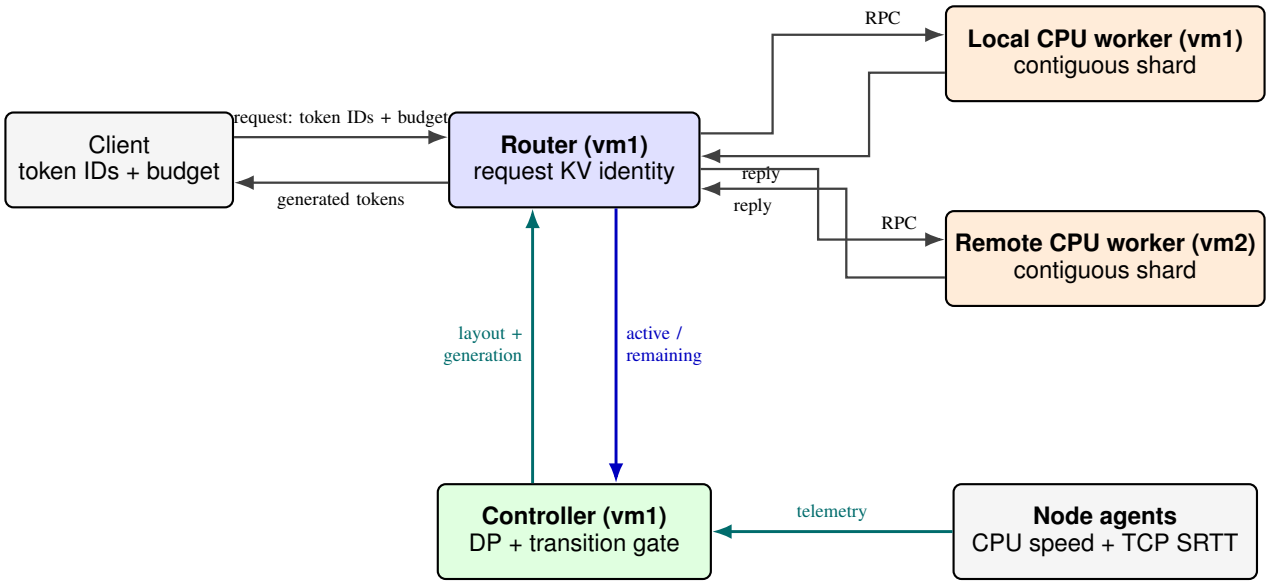


Fig. 1. Shardwise runtime and control flow.

where each remote nonempty worker adds a unit link cost, give a minimum bottleneck $B = 4$ with $P = 11$, for an envelope cost of $\max(4, 5.5) = 5.5$ at $Q = 2$. Two active stages give $B = 5$ and $P = 10$, for cost 5. The revised planner enumerates candidate caps b, solves a sum DP restricted to stages no larger than b, and compares the score before and after the first crossing $b \geq P(b)/Q$. This yields the exact optimum of the restricted model using $O(\log(WL^2))$ constrained DPs. The exactness claim concerns the model only.

C. Payback and finite cohorts

Voluntary moves must first beat the current objective by more than 15% and clear a 30 s cooldown. Recovery moves bypass both checks. The historical gate estimates transition time as

$$T = F + p\,C\,L, \tag{4}$$

where F is fixed reconfiguration cost, C the configured context, and p the replay cost per token-layer. This prices the entire chain, including unmoved layers. A single configured C does not sum simultaneous request histories. The revision adds a live-context forecast $F + p\sum_r C_r L$ over the accepted prompt and output tokens of each active request. It estimates serial replay work only, so overlap, weights, serialization and orchestration remain uncalibrated. When live context is unavailable, voluntary moves are held and recovery is still permitted.

For current and candidate per-token costs $j_c > j_n$, horizon H and margin $m \geq 0$, the historical fluid gate accepts a move when

$$\frac{(H - T)^+}{j_n} > (1 + m)\frac{H}{j_c}. \tag{5}$$

For $T > 0$, this holds if and only if $\Delta_m = j_c - (1 + m)j_n > 0$ and $H > Tj_c/\Delta_m$. Capping the horizon at $\min(H, Rj_c)$, where R is the remaining output of active requests, adds the condition $R > T/\Delta_m$. The sensitivity of the payback horizon to the transition estimate is $\partial H_m/\partial T = j_c/\Delta_m$, which grows as the improvement approaches the margin. Marginal moves are therefore the most sensitive to an underestimated T. This is an algebraic statement about a chosen capacity surrogate. It is not a guarantee of session completion, because a finite cohort cannot produce more than its R remaining tokens and concurrency falls as requests finish.

IV. EVALUATION SETUP

A. Testbed and workloads

The primary experiments use Qwen3-0.6B [20], a 28-layer decoder-only model, in FP32 with KV caching. Two Azure VMs each provide two vCPUs and approximately 8 GiB of

TABLE II
PLACEMENT-OBJECTIVE THROUGHPUT COMPARISON.

Condition	Q	Sum	Bottleneck	Preferred
Stable	1	4.029	3.875	Sum (+4.0%)
Stable	2	4.046	7.303	Bottleneck (+80.5%)
120-ms delay	1	4.001	2.419	Sum (+65.4%)
120-ms delay	2	4.040	4.778	Bottleneck (+18.3%)

memory; one worker thread runs per VM. The coordinator VM also hosts the controller and router. The audited record contains 65 completed runs, 939 requests, and 19 request failures. It covers objective selection, demand variation, compute degradation, network delay, worker loss, and recovery.

B. Policies and metrics

Objective runs compare sum and bottleneck placement at $Q \in \{1, 2\}$ under stable links and 120 ms injected delay. Adaptation runs compare static, hysteresis, and payback-gated policies under a 0.5-CPU quota, delay, or worker loss. We report aggregate throughput, completion time, transition duration, and failures. Whole-run throughput includes transition and drain time; fault-phase throughput counts requests completing within the logged fault interval.

C. Baselines and recovery

Resource-matched local checks compare one- and two-thread unsplit serving, two complete-model replicas, and a two-stage pipeline using identical prompts, decoding, and request paths. Recovery tests inject relayout, worker loss, or restart at accepted-token positions 1, 4, and 12. Eighteen cases compare 27 recovered outputs with unsplit greedy-decoding references.

V. RESULTS AND DISCUSSION

A. Objective selection

With two stable concurrent requests, bottleneck placement reaches 7.303 tokens/s against 4.046 for sum placement, an 80.5% gain (Table II). Median completion falls from 15.777 s to 8.739 s. Under 120 ms delay at $Q = 1$, sum placement reaches 4.001 tokens/s against 2.419 for bottleneck placement, a 65.4% gain. The comparison measures objective choice together with different amounts of active compute, not replica-versus-pipeline performance.

In the demand matrix, automatic policies begin in the latency layout and must observe concurrency before moving. At $Q = 2$, fixed throughput placement reaches 7.830 tokens/s, against 5.944 for automatic placement with a fixed horizon (24.1% lower) and 5.729 with a remaining-work horizon. Most of the gap is time spent in the wrong layout plus the transition itself, since the moves occur when the 30 s cooldown expires.

B. Adaptation and migration payback

Table III reports the full adaptation matrix. Under the 0.5-CPU quota, static placement keeps the slowed worker's share and reaches 4.215 tokens/s in the fault phase. Hysteresis reaches 5.269 tokens/s, a 25.0% gain. Over the whole run, the gain shrinks to 6.8% (6.133 against 5.744), and the gate reaches 5.2%. Reporting only the fault phase would overstate the benefit by a factor of 3.7.

TABLE III
ADAPTATION THROUGHPUT AND FAILURES.

Fault	Policy	Phase	Whole	Failures
CPU quota	Static	4.215	5.744	0
CPU quota	Hysteresis	**5.269**	**6.133**	0
CPU quota	Gate	5.267	6.041	0
120-ms delay	Static	5.270	**6.554**	0
120-ms delay	Hysteresis	5.269	6.131	0
120-ms delay	Gate	5.270	6.055	0
Worker loss	Static	0.000	3.271	19
Worker loss	Hysteresis	**4.219**	**5.443**	0
Worker loss	Gate	3.693	5.409	0

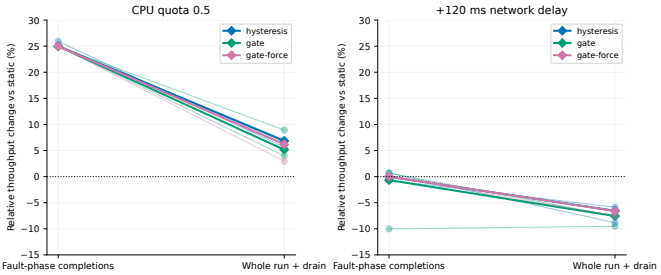


Fig. 2. Adaptive throughput relative to static.

Under 120 ms delay, fault-phase throughput is nearly identical across policies. Over the whole run, the adaptive arms are 6.5–7.6% slower than static placement (6.131 and 6.055 against 6.554 tokens/s for hysteresis and gate). Within this window, their moves cost more than they returned. Under worker loss, static placement completes no request during the fault and records 19 failures across its two runs. Hysteresis and the gate re-place the model on the survivor, record no failures, and sustain 4.219 and 3.693 tokens/s during the fault. The single-device baseline reaches 4.267 tokens/s in the fault-matched phase and 4.158 tokens/s whole-run. The baseline's concurrency level is not recorded in the table, so the fault-phase comparison is indicative only.

No observed median shows a gate advantage over hysteresis. With two or three repetitions, the differences between them are not evidence of a loss either. The defensible finding is that a reasonable cost gate did not show better serving in this record.

C. Forecasts and transition cost

Grouping disrupted requests by executed decision yields 34 transitions. Sixteen occur under compute faults, averaging 5,382 ms (SD 1,839), and 18 under delay faults, averaging 3,948 ms (SD 598). The profile forecast is 3,054 ms. Measurements exceed the forecast by 76.2% and 29.3%. Figure 3 shows that the observed durations are spread widely: compute-fault durations group near 3.4 s and again near 6.9 s, so one

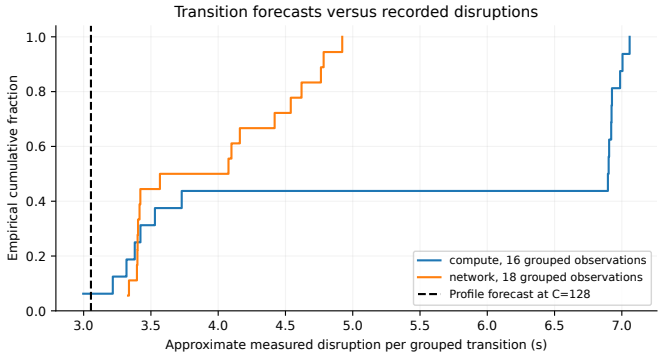


Fig. 3. Transition duration versus forecast.

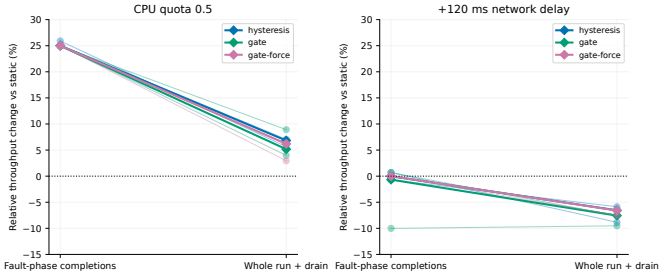


Fig. 4. Resource-matched local throughput.

correction factor describes them poorly. The 34 groups are inferred from request disruptions, because transition identifiers were not logged. Because the payback horizon scales with T, moves that pass the gate can fail to repay.

D. Budget horizons and renewing work

In the remaining-work demand run, four eligible candidates near 23, 25, 27 and 29 s were rejected as the recorded budget fell through 32, 23, 14 and 5 tokens. New closed-loop admissions then raised the budget to 127 tokens, and the move executed at 31.0 s. In the other repeat, the move executed at 23.0 s with 33 remaining tokens. The rule behaves as designed, but in a continuing service the cohort it budgets is replaced before the decision matters. The outcome is a delayed move rather than an avoided one. At $Q = 1$, automatic and fixed-horizon policies differ by under 4%, which is within run-to-run variation.

E. Telemetry diagnostic

In a synthetic loopback experiment with 50 ms service and four concurrent callers, the mean call-minus-compute residual is 149.98 ms. Of this, 148.80 ms is time spent waiting for the worker, and 1.18 ms remains. Queueing therefore accounts for 99.2% of the residual. A controller that priced links from these timings would read congestion as network cost and could move layers for the wrong reason. The experiment is synthetic and is not an Azure or home-network measurement. Client-side admission control removes the contamination from the estimate but does not improve the user's experience: mean completion rises from 200.1 ms to 204.5 ms.

F. Resource-matched local observations

All 32 two-CPU local conditions complete with zero request failures and exact reference matches. At context 32 with $Q = 2$, pipeline and replica arms achieve 7.572 and 7.853 tokens/s; at $Q = 8$, 8.019 and 7.933. At context 128 with $Q = 2$, they achieve 4.816 and 5.063; at $Q = 8$, 5.114 and 5.057. The one-thread unsplit arm trails both at every concurrency tested (Fig. 4). A suitable replica dispatcher is therefore a competitive alternative in these observations, and a baseline without it would miss this deployment choice. With one repetition per condition and no interval estimates, these differences do not establish superiority, equivalence, or a stable ranking.

The three-CPU matrix retains two HTTP failures in the replica arm. Worker 0 exits with SIGTERM during $Q = 8$, and one request fails there and one in the later $Q = 4$ condition that reuses the runtime. The origin of the signal was not established. Fresh-runtime replica reruns at $Q = 8$ and $Q = 4$ serve 12 requests without failure and keep all workers alive. They do not show that the earlier failures cannot recur, and they do not replace the failed observations.

G. Replay correctness and stalls

All 18 position-controlled recovery cases match the unsplit reference token sequence, including both active requests at concurrency two. The 27 outputs contain no duplicated or omitted tokens relative to that reference. Internally accepted token times show a maximum gap of 10.005 s in the fresh validation pass. Exact output is therefore compatible with a substantial user-visible pause. The experiment makes no claim about automatic detection latency or about streamed token delivery to clients.

H. Packaged recovery demonstration

Figure 5 illustrates the packaged application's recovery behavior. Before the injected worker stop, the 24 transformer layers are divided between two local worker processes. After worker 2 is stopped, the controller assigns all layers to worker 1 and subsequent requests continue on the surviving process. This demonstration uses Qwen2.5-0.5B-Instruct on one CPU host and is included only to illustrate control behavior; it is not part of the Qwen3 benchmark results and does not represent an inter-machine network experiment.

The combined evidence shows that repartitioning is most valuable for objective selection under sustained concurrency and for availability after worker loss. Its benefit is less consistent for temporary degradation because transition and drain costs reduce the whole-run gain. Output correctness alone is also insufficient for recovery evaluation: every replayed output matched the reference, but token delivery could still pause for approximately ten seconds. Shardwise therefore treats voluntary performance-driven movement differently from recovery-driven movement. Voluntary moves require an expected payback, whereas worker-loss recovery prioritizes availability.

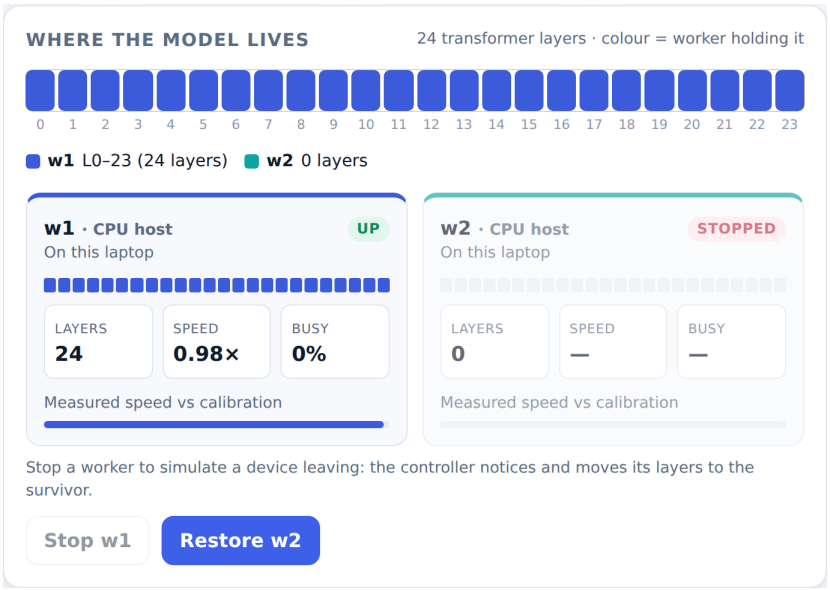


Fig. 5. Packaged recovery before and after worker loss.

I. Limitations

The results are descriptive because most conditions contain only one to three independent repetitions. The primary testbed uses two similarly configured cloud VMs and one 0.6B CPU model; hardware heterogeneity is injected rather than naturally observed. Real-model experiments cover $Q \in \{1, 2\}$, while higher-concurrency observations come from a supporting local experiment. The resource-matched baseline has one run per condition and therefore provides functional evidence rather than a statistically stable ranking. Coordinator failure and streamed client delivery remain outside this evaluation.

VI. Conclusion

The audited two-VM record shows that objective choice, transition accounting and failure recovery matter, while the payback gate shows no demonstrated advantage over hysteresis. The revised runtime corrects a finite-concurrency planner error and exposes replay histories, queue time and token stalls. Position-controlled checks establish replay correctness only for the schedules tested. The one-block local checks show why replicas belong in the baseline set, but they leave stable performance rankings unresolved. This is an empirical study of conditions and pitfalls, and broader deployment and external comparisons remain necessary.

References

[1] Y. Huang *et al.*, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 103–112.

[2] A. Borzunov *et al.*, "Petals: Collaborative inference and fine-tuning of large models," in *Proc. 61st Annual Meeting of the ACL (Vol. 3: System Demonstrations)*, 2023, pp. 558–568. doi:10.18653/v1/2023.acl-demo.54.

[3] M. Zhang, X. Shen, J. Cao, Z. Cui and S. Jiang, "EdgeShard: Efficient LLM inference via collaborative edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13119–13131, 2025. doi:10.1109/JIOT.2024.3524255.

[4] S. Ye *et al.*, "Galaxy: A resource-efficient collaborative edge AI system for in-situ transformer inference," in *IEEE INFOCOM*, 2024, pp. 1001–1010. doi:10.1109/INFOCOM52122.2024.10621342.

[5] Z. Li, W. Feng, M. Guizani and H. Yu, "TPI-LLM: Serving 70B-scale LLMs efficiently on low-resource mobile devices," *IEEE Transactions on Services Computing*, vol. 18, no. 5, pp. 3321–3333, 2025. doi:10.1109/TSC.2025.3596892.

[6] Z. Li *et al.*, "prima.cpp: Fast 30–70B LLM inference on heterogeneous and low-resource home clusters," in *ICLR*, 2026. arXiv:2504.08791.

[7] B. Sun *et al.*, "Llumnix: Dynamic scheduling for large language model serving," in *USENIX OSDI*, 2024.

[8] Y. Fu *et al.*, "ServerlessLLM: Low-latency serverless inference for large language models," in *USENIX OSDI*, 2024.

[9] X. Miao *et al.*, "SpotServe: Serving generative large language models on preemptible instances," in *Proc. ASPLOS*, 2024.

[10] Y. Lin, B. Chen, X. Zhang, C. Xu and K. Ye, "DynoPipe: Heterogeneous edge-cloud LLM serving with dynamically orchestrated pipeline boundaries," in *Proc. ACM/IEEE ISCA*, 2026, pp. 969–984. doi:10.1109/isca66397.2026.00077.

[11] T. Sun, P. Wang and F. Lai, "DiSCo: Device-server collaborative LLM-based text streaming services," in *Findings of ACL*, 2025, pp. 14259–14277. doi:10.18653/v1/2025.findings-acl.734.

[12] Z. Wang *et al.*, "STAR: Decode-phase rescheduling for LLM inference," in *Proc. HPDC*, 2026. doi:10.1145/3806645.3807813.

[13] Z. Chen *et al.*, "ShadowLLM: Resource-efficient hot standby for heterogeneous edge LLM serving," in *Proc. ICDCS*, 2026, pp. 469–479. doi:10.1109/2575-8411.2026.00051.

[14] X. Guo, Q. Jiang, Y. Shen, A. D. Pimentel and T. Stefanov, "EASTER: Learning to split transformers at the edge robustly," *IEEE TCAD*, vol. 43, no. 11, pp. 3626–3637, 2024. doi:10.1109/TCAD.2024.3438995.

[15] S. H. Bokhari, "Partitioning problems in parallel, pipelined, and distributed computing," *IEEE Transactions on Computers*, vol. 37, no. 1, pp. 48–57, 1988. doi:10.1109/12.75137.

[16] D. Kafetzis, R. Khalili and I. Koutsopoulos, "Large language model partitioning for low-latency inference at the edge," 2025. arXiv:2505.02533.

[17] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. SC*, 2015, pp. 73:1–73:12. doi:10.1145/2807591.2807644.

[18] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. doi:10.1287/opre.9.3.383.

[19] M. Reiser and S. S. Lavenberg, "Mean-value analysis of closed multichain queuing networks," *Journal of the ACM*, vol. 27, no. 2, pp. 313–322, 1980.

[20] Qwen Team, "Qwen3 technical report," 2025. arXiv:2505.09388.